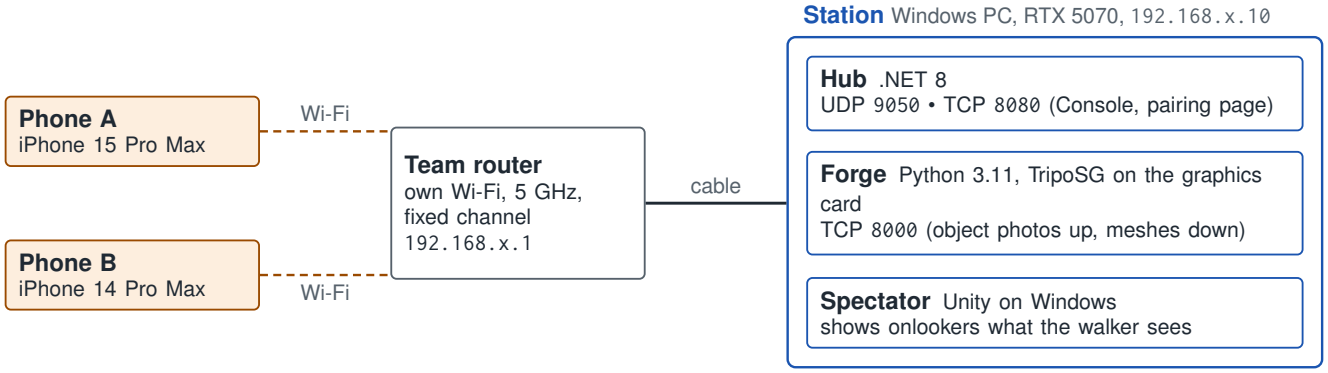
ATLAS: setting up the Station PC

From a bare Windows PC with an RTX 5070 to a Station the phones can reach, with the graphics card proven.

What the Station is

The Station is the Windows PC with the RTX 5070. On the team's own Wi-Fi router it runs three programs for the phones. It cannot be a Mac: the furniture generator needs an NVIDIA card. The phone app is complete without the Station, so a Station that fails costs the mirror, the live tuning and the generated furniture, and nothing else.



What this guide covers

Today and later. Parts 1 to 8 can be done today: the machine, the tools, the graphics card, the network and Unity. Part 9 lists what to run once lane D has written the Hub, the Forge and the Spectator; none of that code exists yet.

	Part	Done when
1	**Prepare Windows**	Windows is up to date, never sleeps on mains power, and you know whether its disk is encrypted
2	**Install the tools**	Git, .NET 8, Python 3.11, the C++ compiler and the CUDA compiler each print their version
3	**Graphics driver**	`nvidia-smi` names the RTX 5070
4	**Get the repository**	`C:\atlas\station` holds `hub`, `forge` and `pipelines`
5	**PyTorch on the graphics card**	The test prints `True` and the card's name. This is task **P1-X03** of the plan
6	**TripoSG smoke test**	One photo becomes a mesh on this card, also with no internet; seconds and video memory recorded
7	**Network**	Both phones open a page served from the Station's fixed address (**P1-X04**, the Station's half)
8	**Unity**	The ATLAS project opens in Unity 6000.3.25f1
9	**When the code lands**	Hub, Forge and Spectator run and a phone pairs
10	**Before every session**	A checklist
11	**If something fails**	A table of symptoms and fixes

Before you start

- **The PC:** Windows 11, 64-bit, an account with administrator rights, the monitor plugged into the RTX 5070. Keep 60 GB of disk free. The known downloads are PyTorch (3.3 GB), the CUDA Toolkit (3.4 GB) and the TripoSG weights (8.8 GB); the C++ compiler and Unity take several GB each.
- **Internet** for Parts 1 to 6 and 8. The team router has none, so do those parts on a normal connection.
- **For Part 7:** the team's router, a network cable and both phones.
- **Accounts:** GitHub with access to the repository, and a Unity account.

How to read the commands

- Grey blocks are typed into PowerShell (the Terminal app). Copy them out of this PDF: they paste exactly as printed. A block of several lines can be pasted in one go; Terminal asks once whether to paste several lines.
- ADMINISTRATOR means a terminal opened with a right-click on the Start button › Terminal (Admin).
- In `192.168.x.10`, the first three numbers are your router's. Replace the `x` wherever it appears.
- Two folders are used throughout: `C:\atlas` (the repository) and `C:\triposg` (the mesh generator).

1 Prepare Windows

Done when: Windows is up to date, never sleeps on mains power, and you know whether its disk is encrypted.

1.1 Update Windows first. Settings › Windows Update: install everything and restart until nothing is pending. An update that arrives later restarts the PC in the middle of a session.

1.2 Check the installer tool. Part 2 uses `winget`.

```
winget --version
```

You should see: a version number. If the command is not found, run the line below and open a new terminal.

```
Add-AppxPackage -RegisterByFamilyName -MainPackage Microsoft.DesktopAppInstaller_8wekyb3d8bbwe
```

1.3 Allow long paths. ADMINISTRATOR Unity and Python packages make paths longer than the old Windows limit of 260 characters.

```
$fs = "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem"
New-ItemProperty -Path $fs -Name LongPathsEnabled -Value 1 -PropertyType DWORD -Force
```

1.4 Allow your own scripts to run. Python environments are switched on by a script. Answer `Y`.

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

1.5 Never sleep on mains power. ADMINISTRATOR A sleeping PC stops the Hub and the Forge. A locked screen does not.

```
powercfg /change standby-timeout-ac 0
powercfg /change hibernate-timeout-ac 0
powercfg /change monitor-timeout-ac 0
```

Check in Settings › System › Power: sleep and screen-off read *Never* when plugged in.

1.6 See whether the disk is encrypted. ADMINISTRATOR The Hub will keep motion logs of walkers, which the plan treats as health data.

```
manage-bde -status C:
```

You should see: `Protection Status: Protection On`. If it reads `Off`, turn on Settings › Privacy & security › Device encryption (named BitLocker on Windows Pro). If Windows offers neither, note it on the record sheet: the Hub's logs then go into an encrypted container before anyone from outside the team is logged.

1.7 Restart the PC so the long-path setting reaches every program.

2 Install the tools

Done when: the five checks in step 2.6 each print the line shown.

2.1 Git, .NET 8 and Python 3.11. Answer `Y` to the agreement questions. Git for Windows brings Git LFS with it.

```
winget install --id Git.Git -e
winget install --id Microsoft.DotNet.SDK.8 -e
winget install --id Python.Python.3.11 -e
```

2.2 The C++ compiler. It is needed once, in Part 6, to build one part of TripoSG. It must be the 2022 edition: the CUDA compiler of the next step does not accept Visual Studio 2026.

```
$vs = "--passive --wait --norestart --add Microsoft.VisualStudio.Workload.VCTools --includeRecommended"
winget install --id Microsoft.VisualStudio.2022.BuildTools -e --override $vs
```

2.3 The CUDA Toolkit 12.8. It brings `nvcc`, the compiler for code that runs on the graphics card. PyTorch itself does not need it; the TripoSG build in Part 6 does. Download the installer (3.4 GB) and start it:

```
$f = "cuda_12.8.1_572.61_windows.exe"
curl.exe -L -o $HOME\Downloads\$f https://developer.download.nvidia.com/compute/cuda/12.8.1/local_installers/$f
& "$HOME\Downloads\$f"
```

Choose *Custom (Advanced)* and untick everything except *CUDA*. Above all, untick *Driver components*: the installer carries driver 572.61, which is older than the first driver that lists the RTX 5070 (572.70), and it must not replace yours. For the same reason, do not use its Express option and do not install the toolkit through `winget`: both install every component.

2.4 Unity Hub , for Part 8.

```
winget install --id Unity.UnityHub -e
```

2.5 Restart the PC if an installer asked for it. Otherwise close the terminal and open a new one, so it sees the new programs.

2.6 Check all five.

```
git --version
git lfs version
dotnet --list-sdks
py -3.11 --version
nvcc --version
```

You should see:

```
git version 2.5x.x.windows.x
git-lfs/3.x.x (GitHub; windows amd64; go 1.xx)
8.0.4xx [C:\Program Files\dotnet\sdk]
Python 3.11.9
... Cuda compilation tools, release 12.8, V12.8.xx
```

2.7 Set Git up for this user.

```
git config --global core.longpaths true
git lfs install
```

Two traps. Typing `python` outside an environment may open the Microsoft Store: always type `py -3.11`. If a newer .NET is already installed, keep it and install 8 as well; `dotnet -list-sdks` must list an `8.0` line.

3 Graphics driver

Done when: `nvidia-smi` names the RTX 5070 and shows a driver of 572.70 or newer.

3.1 Install the newest GeForce driver, also when a driver is already there. Use the NVIDIA app, or `nvidia.com/drivers`: GeForce RTX 5070, Windows 11, Game Ready. Restart when it asks. Doing this after the CUDA Toolkit also puts things right if the toolkit's installer did replace the driver.

3.2 Check the card.

```
nvidia-smi
```

You should see: a table with `NVIDIA GeForce RTX 5070`, a `Driver Version` of 572.70 or higher, and about 12,000 MiB of memory. Write the driver version on the record sheet.

3.3 Make a full graphics memory fail fast. NVIDIA Control Panel › Manage 3D settings › Global Settings › *CUDA - Sysmem Fallback Policy* › *Prefer No Sysmem Fallback* › Apply. Without it, a job that runs out of video memory carries on from system memory, many times slower, and reports no error.

4 Get the repository

Done when: `C:\atlas\station` holds `hub`, `forge` and `pipelines`.

4.1 Clone it to a short path. A browser window asks you to sign in to GitHub the first time.

```
git clone https://github.com/notwinston/StormHacks_2026_Blueberries.git C:\atlas
cd C:\atlas
git lfs pull
```

4.2 Check.

```
dir C:\atlas\station
```

You should see: three folders: `forge`, `hub`, `pipelines`.

5 PyTorch on the graphics card

Done when: the test prints `True` and the card's name. With .NET 8 from Part 2, that is task **P1-X03** of the plan.

5.1 Make the Forge's Python environment. The folder `.venv` is ignored by Git.

```
cd C:\atlas\station\forge
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel ninja
```

You should see: the prompt now starts with `(.venv)`. Every later Python command needs that prefix: after opening a new terminal, run the third line again.

5.2 Install PyTorch built for CUDA 12.8. Use this line as printed. The chooser on `pytorch.org` no longer lists 12.8, and its older choice (12.6) does not support the 5070.

```
pip install torch==2.7.1 torchvision==0.22.1 --index-url https://download.pytorch.org/whl/cu128
```

5.3 Test it.

```
python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available())"
python -c "import torch; print(torch.cuda.get_device_name(0), torch.cuda.get_device_capability(0))"
python -c "import torch; x = torch.rand(4000, 4000, device='cuda'); print(float((x @ x).mean()))"
```

You should see:

```
2.7.1+cu128 12.8 True
NVIDIA GeForce RTX 5070 (12, 0)
a number close to 1000, and no warning
```

If it fails A version that ends in `+cpu`, `False`, or a warning that `sm_120 is not compatible` all mean the wrong build was installed. Remove it and repeat step 5.2 exactly, including `--index-url`:

```
pip uninstall -y torch torchvision
```

6 TripoSG smoke test

Done when: one photo becomes a mesh of about 5,000 triangles on this card, also with the network unplugged, and the seconds and the peak video memory are on the record sheet.

Why this part matters. TripoSG is the generator inside the Forge. One of its parts, `diso`, has no ready-made Windows build and is compiled on this PC in step 6.3. Nobody has reported that build on an RTX 50 card, so this is the step most likely to fail. The plan marks it "verify"; this part is that check, and it also measures the seconds per object the plan still lacks.

6.1 Get TripoSG at the pinned version, outside the repository.

```
git clone https://github.com/VAST-AI-Research/TripoSG.git C:\triposg
cd C:\triposg
git checkout fc5c40990181e2a756c4e0b1c2f4d6b5202faf8c
```

6.2 Install its Python packages into the environment of Part 5 (the prompt shows `(.venv)`). Do not use TripoSG's own `requirements.txt`: it asks for a NumPy that has no Python 3.11 build, and it leaves packages out. These are the versions of April 2025, when TripoSG was last changed; newer ones break it.

```
pip install numpy==1.26.4 scipy==1.17.1 scikit-image==0.26.0 opencv-python==4.11.0.86
pip install diffusers==0.32.2 transformers==4.51.1 huggingface_hub==0.30.2 tokenizers==0.21.1 safetensors==0.5.3
pip install peft==0.15.1 accelerate==1.6.0 einops==0.8.2 omegaconf==2.3.1 trimesh==4.12.2
pip install jaxtyping==0.3.11 typeguard==4.6.0 pymeshlab==2025.7.post1
pip check
python -c "import torch; print(torch.__version__)"
```

You should see: `No broken requirements found.` and then `2.7.1+cu128`: PyTorch is still the build of Part 5.

6.3 Build `diso`. This runs the C++ compiler and `nvcc` from Part 2. Use a normal PowerShell window, not a Visual Studio developer prompt. The first three lines point the build at the 12.8 toolkit and at the 5070.

```
$env:CUDA_HOME = "C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.8"
$env:FORCE_CUDA = "1"
$env:TORCH_CUDA_ARCH_LIST = "12.0"
pip install diso==0.1.4 --no-build-isolation --no-cache-dir
python -c "import torch; from diso import DiffDMC; print('diso ok')"
```

You should see: `Successfully installed diso-0.1.4`, then `diso ok`.

6.4 First run. It downloads 8.8 GB of weights into `C:\triposg\pretrained_weights`, once.

```
$img = "assets/example_data/hjswed.png"
python -m scripts.inference_triposg --image-input $img --output-path out.glb
```

You should see: `Mesh saved to out.glb` as the last line.

6.5 Timed run with the triangle limit. First start this in a second terminal and leave it running; it prints the video memory in use once a second.

```
nvidia-smi --query-gpu=memory.used,memory.total --format=csv -l 1
```

Then paste these three lines together in the first terminal. The last one prints the seconds.

```
$t = Get-Date
python -m scripts.inference_triposg --image-input $img --faces 5000 --output-path out_5000.glb
((Get-Date) - $t).TotalSeconds
```

Write down the seconds and the highest memory figure. The time includes loading the model, which the Forge does only once.

6.6 Check the mesh.

```
python -c "import trimesh; m = trimesh.load('out_5000.glb', force='mesh'); print(len(m.faces), m.extents)"
```

You should see: a count of about 5000, then three sizes. To look at it, drag the file into a glTF viewer in the browser, for example `sandbox.babylonjs.com`: it must be a recognisable object.

6.7 Run it with no internet. Pull the network cable or turn Wi-Fi off, then:

```
$env:HF_HUB_OFFLINE = "1"
python -m scripts.inference_triposg --image-input $img --faces 5000 --output-path out_offline.glb
```

It must finish as before, with no download. The demo runs on a router with no internet.

6.8 Save what worked, then commit the file: it is the first version of the Forge's lock file.

```
pip freeze | Set-Content C:\atlas\station\forge\requirements.lock.txt
```

If this part fails. Copy the first error line and give it to lane D; Part 11 lists the known ones. The ways out, in order: rebuild `diso` in a fresh environment; run TripoSG without `diso` on its slower built-in surface extraction (a small code change, untested); run the Forge under WSL2; use the hosted service of open item 8, or the built-in furniture set. The app does not depend on the Forge.

For lane D. **Licences:** TripoSG is MIT, but `diso` is CC BY-NC 4.0 and TripoSG's own background remover (RMBG-1.4) is non-commercial. Enter all three in the licence ledger. **Background removal:** the current `rembg` needs NumPy 2.3 or newer, which clashes with the set above, and its default model is now the non-commercial `bria-rmbg`; name the model (`u2net`) and settle NumPy in the lock file. **Working folder:** TripoSG looks for `pretrained_weights` under the folder it is started from, so the Forge must start it from `C:\triposg`. **Versions:** do not upgrade the set of step 6.2; `transformers` 4.57 and newer stop TripoSG from loading.

7 Network

Done when: both phones open the Station's test page at its fixed address, on port 8080 and on port 8000. That is the Station's half of task **P1-X04**.

7.1 Set the router up, once. Give it a Wi-Fi name and password of your own. On the 5 GHz band, fix the channel: 36, 40, 44, 48, 149, 153, 157 or 161. Never "Auto", and never a channel from 52 to 144, which the router must leave when it detects radar. Set the channel width to 20 MHz: the link needs little bandwidth and a narrow channel is steadier in a crowded hall. Channel 44 or 149 is the best pick, because the phones' own AirDrop radio rests there. The router needs no internet cable.

7.2 Cable the PC to the router (a LAN port) and read the adapter's name, its address and its hardware address.

```
Get-NetAdapter -Physical | Format-Table Name, Status, MacAddress, LinkSpeed
Get-NetIPAddress -AddressFamily IPv4 | Format-Table InterfaceAlias, IPAddress
```

The wired adapter is usually named `Ethernet`. If yours differs, use your name in the commands below.

7.3 Give the Station the fixed address `192.168.x.10`. In the router's own page, reserve that address for the hardware address you just read (the setting is called DHCP reservation or static lease). Then:

```
ipconfig /renew
ipconfig
```

You should see: `IPv4 Address . . . : 192.168.x.10` under the wired adapter.

7.4 Mark the network as private. ADMINISTRATOR Windows blocks incoming connections on a network it calls public.

```
Get-NetConnectionProfile
Set-NetConnectionProfile -InterfaceAlias "Ethernet" -NetworkCategory Private
```

7.5 Open the three ports. ADMINISTRATOR These are the plan's three firewall rules, limited to private networks.

```
$allow = @{ Direction = "Inbound"; Action = "Allow"; Profile = "Private" }
New-NetFirewallRule @allow -DisplayName "ATLAS Hub UDP" -Protocol UDP -LocalPort 9050
New-NetFirewallRule @allow -DisplayName "ATLAS Hub HTTP" -Protocol TCP -LocalPort 8080
New-NetFirewallRule @allow -DisplayName "ATLAS Forge" -Protocol TCP -LocalPort 8000
Get-NetFirewallRule -DisplayName "ATLAS*" | Format-Table DisplayName, Enabled, Direction, Action, Profile
```

You should see: three rows, each `True`, `Inbound`, `Allow`, `Private`.

7.6 Serve a test page. If Windows asks whether Python may be reached from networks, allow private networks only.

```
mkdir C:\atlas-test
cd C:\atlas-test
Set-Content index.html "<h1>ATLAS Station is reachable</h1>"
py -3.11 -m http.server 8080 --bind 0.0.0.0
```

7.7 ON EACH PHONE Join the team's Wi-Fi. iOS notices that it has no internet and may offer to use mobile data: decline and stay on the Wi-Fi. Open Safari at `http://192.168.x.10:8080`.

You should see: the heading *ATLAS Station is reachable*.

7.8 Repeat for the Forge's port. Stop the server with Ctrl+C, start it on 8000, and open `http://192.168.x.10:8000` on both phones.

```
py -3.11 -m http.server 8000 --bind 0.0.0.0
```

Stop it with Ctrl+C when both phones have shown the page. The UDP port 9050 cannot be tested with a browser; it is proven when the Hub runs (Part 9).

7.9 ON EACH PHONE Before every session: Settings › General › AirDrop › Receiving Off, and Settings › General › AirPlay & Continuity › Handoff off. Both make the phone's radio hop channels, which shows as stutter on the link.

If the PC loses its internet when the router is cabled: Windows prefers the cable, and the team router leads nowhere. This happens when the PC also uses Wi-Fi on another network for its internet. In an administrator terminal, make the cable the last choice, then check that downloads work again.

```
Set-NetIPInterface -InterfaceAlias "Ethernet" -AddressFamily IPv4 -InterfaceMetric 500
```

If the router cannot reserve addresses: set the address on the PC instead, in an administrator terminal, with your router's numbers in place of `x`. Pick an address outside the range the router hands out by itself.

```
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.x.10 -PrefixLength 24
```

8 Unity

Done when: the ATLAS project opens in Unity 6000.3.25f1 with no red error in its Console window.

8.1 Open Unity Hub and sign in with your Unity account. If the Hub shows no licence, add the free Personal one (account menu › Manage licenses › Add).

8.2 Install exactly the project's Editor version. This opens the Hub's install dialog for that version.

```
Start-Process "unityhub://6000.3.25f1/e1dba0a9aba4"
```

Building for Windows needs no extra module, so every optional box can stay unticked. If the link does nothing: Hub › Installs › Install Editor › Archive, and pick 6000.3.25f1 there.

8.3 Register Git LFS and Unity's merge tool for this clone. The script needs the Editor of the step before.

```
cd C:\atlas
& "C:\Program Files\Git\bin\bash.exe" tools/setup_git.sh
```

You should see: `Git LFS and UnityYAMLMerge (6000.3.25f1) are set up for this clone.`

8.4 Open the project. Hub › Add › Add project from disk › `C:\atlas\ATLAS`, then open it with 6000.3.25f1. If the Hub proposes another Editor version, cancel: a different version rewrites project files. The first opening imports every asset.

8.5 Check. Window › General › Console shows no red error. If it does, send the first one to lane A: the project has not been opened on Windows before.

9 When the code lands

Done when: the Hub, the Forge and the Spectator run, and a phone pairs by QR code.

Waits for code Nothing in this part can be done today. The Hub, the Forge service and the Spectator are tasks **P1-D02**, **P1-D08** and **P1-D05** of lane D. The commands below are the plan's (Appendix C.7); when a folder has a README, the README wins.

9.1 Hub. It listens on UDP 9050 and serves the Console on TCP 8080.

```
cd C:\atlas
dotnet run --project station/hub
```

Open `http://localhost:8080` for the Console. If the Hub prints `no packets received` ten seconds after a phone tried to connect, the firewall or the network category is wrong: steps 7.4 and 7.5.

9.2 Forge. Start it from the environment of Part 5, after installing the Forge's lock file.

```
cd C:\atlas\station\forge
.\.venv\Scripts\Activate.ps1
uvicorn forge.app:app --host 0.0.0.0 --port 8000
```

Open `http://localhost:8000/health`. It reports `ok`, `gpu`, `queue`, `pipelineVer` and `debugKeep`. `debugKeep` must be `false` whenever someone from outside the team is scanned: it means object photos are kept on disk.

9.3 Pair a phone. Open `http://localhost:8080/pair` on the Station: it shows a QR code. Scan it with the phone's Camera app. The app asks once for permission to use the local network: allow it, with the phone in your hand.

9.4 Spectator. Run the Unity Editor in play mode, or the Windows build made with the `ATLAS_SPECTATOR` symbol. It joins the Hub as a spectator.

9.5 Logs. The Hub's `logs` folder must sit on the encrypted disk of step 1.6.

9.6 One program owns the graphics card. The Forge runs one job at a time. Do not run the TripoSG test of Part 6 while the Forge is up, and close other programs that use video memory.

9.7 Lung data pipeline. `station/pipelines` has its own environment and README; it is being written on the branch `p2-lung-data`.

10 Before every session

Done when: every box is ticked.

- ☐ The PC is on mains power and does not sleep (step 1.5).
- ☐ Windows Update shows nothing pending, and updates are paused (Settings › Windows Update › Pause).
- ☐ Do Not Disturb is on (Settings › System › Notifications).
- ☐ The cable goes to the team router; `ipconfig` shows `192.168.x.10`; the network is Private (step 7.4).
- ☐ The router is on its fixed channel.
- ☐ With no internet: `$env:HF_HUB_OFFLINE = "1"` is set in the terminal that starts the Forge.
- ☐ Started in this order: Hub, Forge, Spectator, then the Console in the browser.
- ☐ `/health` reports `ok`, and `debugKeep` is `false`.
- ☐ `nvidia-smi` shows little video memory in use before the first job.
- ☐ Both phones: on the team Wi-Fi, AirDrop and Handoff off, link dot green.
- ☐ Everyone knows the rule: if the Station fails, carry on. The app is complete without it and the furniture falls back to the built-in set.

11 If something fails

Symptom	Cause	Fix
`winget` is not recognised	App Installer is not registered yet	Step 1.2
`Activate.ps1`: running scripts is disabled	PowerShell's default policy	Step 1.4
`python` opens the Microsoft Store	A Windows alias stands in for Python	Type `py -3.11`, or activate the environment first
The PyTorch version ends in `+cpu`, or the test prints `False`	The plain build was installed	Uninstall it and repeat step 5.2 with `--index-url`
Warning: `sm_120 is not compatible`	A build for CUDA 12.6 or older	The same
`diso`: Microsoft Visual C++ 14.0 or greater is required	The C++ compiler is missing, or this terminal is older than its install	Step 2.2, then a new terminal. Still failing: open the Visual Studio Installer and tick *Desktop development with C++*
`diso`: the VC environment is activated but `DISTUTILS_USE_SDK` is not set	The build was started from a Visual Studio developer prompt	Use a normal PowerShell window
`diso`: invalid command `bdist_wheel`	The `wheel` package is missing	The fourth line of step 5.1
`diso`: the detected CUDA version mismatches the version used to compile PyTorch	Another CUDA Toolkit was found first	Check that the folder in the first line of step 6.3 exists; install the 12.8 toolkit (step 2.3)
`diso`: `nvcc` reports an unsupported Visual Studio version	Visual Studio 2026 is installed	Install the 2022 Build Tools of step 2.2 and remove 2026
`diso`: No module named torch, during the build	`--no-build-isolation` was left out	Repeat step 6.3 as printed
TripoSG ends with `'NoneType' object has no attribute 'astype'`	`diso` failed on the graphics card; the real error is the `CUDA error` line printed above it	`pip uninstall -y diso`, then step 6.3 in a new terminal. If it comes back, give that line to lane D
TripoSG: unexpected keyword argument `offload_state_dict`	A newer `transformers` was installed	Repeat the second line of step 6.2
`DLL load failed while importing _C`	`diso` was built against another PyTorch or CUDA	Delete `.venv`, redo Part 5 and steps 6.2 and 6.3
No module named `triposg`, or the weights download again	TripoSG was started from the wrong folder	`cd C:\triposg` and use `python -m scripts.inference_triposg`
TripoSG is very slow and prints no error	Video memory is full and Windows spilled into system memory	Step 3.3; close Unity and browsers; watch `nvidia-smi`
A phone cannot open the test page	The network is Public, a rule is missing, the address is wrong, or the phone left the Wi-Fi	Steps 7.3 to 7.5; on the phone, check it is still on the team Wi-Fi
The PC lost its internet when the router was cabled	Windows prefers the cable	The first box after step 7.9
A phone never connects to the Hub	In this order: the local-network permission was refused; a firewall rule is missing; the server listens on `localhost` only	Phone: Settings › Privacy & Security › Local Network › ATLAS on. PC: step 7.5. Server: it must listen on `0.0.0.0`
Unity Hub proposes another Editor version	The exact version is not installed	Cancel; step 8.2
Pink or missing assets in Unity	The large files were not fetched	`git lfs pull` in `C:\atlas`
The Hub and the Forge stopped by themselves	The PC went to sleep	Step 1.5

Record sheet

Fill this in as you go, then copy it into `docs/versions.md` under "Station PC". The plan asks for the installed numbers, and two of them are still open questions: whether TripoSG runs on this card, and how long one object takes.

Windows edition and version	_____	PyTorch and torchvision	_____
Disk encrypted (step 1.6)	_____	TripoSG commit (first 8 characters)	_____
GeForce driver	_____	`diso` built and imported	_____
.NET SDK	_____	Seconds for one object, 5,000 triangles	_____
Python	_____	Peak video memory (MiB)	_____
C++ Build Tools	_____	Ran with no internet	_____
CUDA Toolkit (`nvcc`)	_____	Station address	_____
Unity Editor	_____	Router channel and width	_____
Both phones opened port 8080 and 8000	_____	Date, and who did it	_____